

Task 2.2

Convolutional Neural Network (CNN) model

For the weather and pleasant weather data sets, I chose to use a CNN model at first, as it would likely take less time to process the large volume of data than a Recurring Neural Network (RNN) would. The temporal elements of both datasets being removed might also affect the performance of a RNN model in this case.

Starting Parameters

```
epochs = 30  
batch_size = 16  
n_hidden = 4
```

```
timesteps = len(X_train[0])  
input_dim = len(X_train[0][0])  
n_classes = len(y_train[0])
```

```
model = Sequential()  
model.add(Conv1D(n_hidden, kernel_size=2, activation='relu', input_shape=(timesteps,  
input_dim)))  
model.add(Dense(2, activation='relu'))  
model.add(MaxPooling1D())  
model.add(Flatten())  
model.add(Dense(n_classes, activation='softmax'))
```

```
accuracy: 0.1052 - loss: 1574558.7500
```

Confusion Matrix with Starting Model Parameters

Pred True	BELGRADE	BUDAPEST	DUSSELDORF	HEATHROW	LJUBLJANA	MAASTRICHT	\
BASEL	13	222	140	1192	495	19	
BELGRADE	7	63	2	267	211	0	
BUDAPEST	1	14	0	57	40	0	
DEBILT	0	6	0	42	4	0	
DUSSELDORF	0	3	0	12	2	0	
HEATHROW	0	5	1	47	1	0	
KASSEL	0	1	1	3	3	0	
LJUBLJANA	1	0	1	18	11	0	
MAASTRICHT	0	0	0	5	2	0	
MADRID	0	31	7	159	39	0	
MUNCHENB	0	0	0	3	2	0	
OSLO	0	0	0	1	2	0	
STOCKHOLM	0	0	0	1	0	0	
VALENTIA	0	0	0	1	0	0	

Pred True	MADRID	MUNCHENB	OSLO	STOCKHOLM
BASEL	1126	6	2	467
BELGRADE	451	0	0	91
BUDAPEST	84	0	0	18
DEBILT	17	0	0	13
DUSSELDORF	11	0	0	1
HEATHROW	20	0	0	8
KASSEL	2	0	0	1
LJUBLJANA	29	0	0	1
MAASTRICHT	2	0	0	0
MADRID	218	0	0	4
MUNCHENB	1	0	0	2
OSLO	1	0	0	1
STOCKHOLM	0	0	0	3
VALENTIA	0	0	0	0

This model recognized 10 out of 15 stations. Rerunning the model shows the model recognizing 12, 14 and even 15 stations on different tries.

Adjusting Hyperparameters and final activation layer function

Decreased the batch size to 9, doubled the number of hidden layers to 4 , doubled the size of the dense layer to 8 and increased the kernel size to 3.

accuracy: 0.1123 - loss: 50368612.0000

Doubled the number of hidden layers to 8 and the size of the dense layer to 16.

accuracy: 0.1227 - loss: 160600112.0000

increased the number of hidden layers to 24.

accuracy: 0.1200 - loss: 540405312.0000

Since that did not improve accuracy, changed the number of hidden layers back to 16.

Added a second dense layer after the one in place with the same size (16).

accuracy: 0.1197 - loss: 194700591104.0000

Since the volume of data is very large, I doubled the number of epochs to 60 and doubled the batch size to 18 (the model would otherwise take a very long time to run).

accuracy: 0.1240 - loss: 99357114368.0000

Changing final activation layer function from softmax to sigmoid

Same parameters as before, with the final activation layer having a sigmoid function instead of softmax.

accuracy: 0.6470 - loss: 84387127296.0000

We can see that changing the final activation layer to sigmoid has greatly improved the accuracy of this model. However, the amount of loss is still massive.

We can test the same model with the number of hidden layers increased to 64.

accuracy: 0.6467 - loss: 229174657024.0000

Changing the number of hidden layers back to 32. This time we can increase the batch size to 30 and the number of epochs to 100.

accuracy: 0.6468 - loss: 53314588672.0000

Since this did not improve the accuracy, we can try the model with 50 epochs and a smaller batch size of 9.

accuracy: 0.6469 - loss: 2228606205952.0000

Changed the final activation layer to tanh.

Reduced the size of the first Dense layer in half.

accuracy: 0.0653 - loss: 20.8934

Removed the larger Dense layer.

accuracy: 0.0134 - loss: 24.9629

Increased n_hidden to 64. Increased the size of the Dense layer to 32.

accuracy: 0.2106 - loss: 23.3394

Repeating the above with 50 epochs and batch_size of 15.

accuracy: 0.2938 - loss: 25.8269

Repeating the above with the final activation layer set to ReLU

accuracy: 0.6469 - loss: nan

Reduced the batch size to 9.

accuracy: 0.6469 - loss: nan

Accuracy did not change after 2nd epoch.

Final set of hyperparameters

epochs = 60

batch_size = 9

n_hidden = 64

model = Sequential()

model.add(Conv1D(n_hidden, kernel_size=3, activation='relu', input_shape=(timesteps, input_dim)))

model.add(Dense(32, activation='relu'))

model.add(MaxPooling1D())

model.add(Flatten())

model.add(Dense(n_classes, activation='relu'))

Confusion Matrix for CNN Model using ReLU activation in final layer

accuracy: 0.6469 - loss: nan

180/180 ————— 0s 2ms/step

Pred	BASEL
True	
BASEL	3682
BELGRADE	1092
BUDAPEST	214
DEBILT	82
DUSSELDORF	29
HEATHROW	82
KASSEL	11
LJUBLJANA	61
MAASTRICHT	9
MADRID	458
MUNCHENB	8
OSLO	5
STOCKHOLM	4
VALENTIA	1

With the final activation layer set to ReLU, the model shows 65% accuracy and no loss, but only recognizes one station.

Rerunning the model with the final activation function changed to sigmoid

accuracy: 0.6468 - loss: 1427298304.0000

180/180 ————— 0s

Pred	BASEL	VALENTIA
True		
BASEL	3680	2
BELGRADE	1092	0
BUDAPEST	214	0
DEBILT	82	0
DUSSELDORF	29	0
HEATHROW	82	0
KASSEL	11	0
LJUBLJANA	61	0
MAASTRICHT	9	0
MADRID	458	0
MUNCHENB	8	0
OSLO	5	0
STOCKHOLM	4	0
VALENTIA	1	0

Same accuracy level, massive loss and the confusion matrix shows that the model only recognizes 2 out of the 15 stations.

Rerunning the model with the final activation function changed to tanh

accuracy: 0.3159 - loss: 23.6901

Confusion matrix

180/180		0s 1ms/step					
Pred	BELGRADE	BUDAPEST	DUSSELDORF	LJUBLJANA	MAASTRICHT	SONNBLICK	\
True							
BASEL	1214	9	2297	27	4	5	
BELGRADE	730	1	334	0	0	4	
BUDAPEST	124	0	81	0	0	1	
DEBILT	62	0	19	0	0	0	
DUSSELDORF	20	0	9	0	0	0	
HEATHROW	30	0	49	0	0	0	
KASSEL	5	1	5	0	0	0	
LJUBLJANA	12	0	45	0	0	1	
MAASTRICHT	1	0	6	0	0	0	
MADRID	64	0	376	0	0	1	
MUNCHENB	0	0	7	0	0	1	
OSLO	1	0	3	0	0	0	
STOCKHOLM	1	0	1	0	0	1	
VALENTIA	0	0	1	0	0	0	
Pred	VALENTIA						
True							
BASEL	126						
BELGRADE	23						
BUDAPEST	8						
DEBILT	1						
DUSSELDORF	0						
HEATHROW	3						
KASSEL	0						
LJUBLJANA	3						
MAASTRICHT	2						
MADRID	17						
MUNCHENB	0						
OSLO	1						
STOCKHOLM	1						
VALENTIA	0						

The model only recognized 7 out of 15 stations initially.

Rerunning the model with the final set of parameters and the final activation function set to softmax

accuracy: 0.1246 - loss: 16889231360.0000

Confusion Matrix of CNN model with updated parameters and softmax final activation layer

180/180 ————— 0s 2ms/step

Pred	BASEL	BELGRADE	BUDAPEST	DEBILT	DUSSELDORF	HEATHROW	KASSEL	\
True								
BASEL	3	2037	6	17	6	621	7	
BELGRADE	0	1045	0	0	0	26	0	
BUDAPEST	0	190	0	0	0	17	0	
DEBILT	0	70	0	0	0	11	0	
DUSSELDORF	0	21	0	0	0	5	0	
HEATHROW	0	48	0	0	0	21	0	
KASSEL	0	10	0	0	0	1	0	
LJUBLJANA	0	42	0	0	0	4	0	
MAASTRICHT	0	4	0	0	0	4	0	
MADRID	0	205	0	0	0	99	0	
MUNCHENB	0	6	0	0	0	0	0	
OSLO	0	2	0	0	0	1	0	
STOCKHOLM	0	3	0	0	0	0	0	
VALENTIA	0	0	0	0	0	1	0	

Pred	LJUBLJANA	MAASTRICHT	MADRID	MUNCHENB	OSLO	SONNBLICK	\
True							
BASEL	34	70	781	7	17	11	
BELGRADE	0	0	21	0	0	0	
BUDAPEST	0	0	7	0	0	0	
DEBILT	0	0	1	0	0	0	
DUSSELDORF	0	0	3	0	0	0	
HEATHROW	0	0	13	0	0	0	
KASSEL	0	0	0	0	0	0	
LJUBLJANA	0	0	15	0	0	0	
MAASTRICHT	0	0	1	0	0	0	
MADRID	1	1	152	0	0	0	
MUNCHENB	0	0	2	0	0	0	
OSLO	0	0	2	0	0	0	
STOCKHOLM	0	0	1	0	0	0	
VALENTIA	0	0	0	0	0	0	

Pred	STOCKHOLM	VALENTIA
True		
BASEL	48	17
BELGRADE	0	0
BUDAPEST	0	0
DEBILT	0	0
DUSSELDORF	0	0
HEATHROW	0	0
KASSEL	0	0
LJUBLJANA	0	0
MAASTRICHT	0	0
MADRID	0	0
MUNCHENB	0	0
OSLO	0	0
STOCKHOLM	0	0
VALENTIA	0	0

This model recognized all 15 weather stations initially. Subsequent reruns of the model showed that the model recognized 14 out of 15 stations.

Observations

- Models with a small batch size and larger number of epochs took longer to run.
- A larger number of hidden layers and a larger sized dense layer appeared to improve overall accuracy.
- Since this is a large dataset, an increased kernel size improved the performance of the model.
- The model recognized the most weather stations when the final activation layer function was set to softmax. However, the accuracy of the predictions was very low, at around 12%, with a massive amount of loss.
- Using the sigmoid and ReLU activation functions in the final layer produced the highest level of accuracy, however the confusion matrices show that the model only recognized one or two of the stations. This might be indicative of the model overfitting data from these stations when these activation functions are in place.

So far, it appears that the CNN model using the softmax activation function in its last layer works best when making predictions with this data. Reducing the batch size, increasing the kernel size and increasing the number of hidden layers as well as the size of the dense layer seem to improve the model's performance. While adding another dense layer of the same size did not seem to improve the model accuracy, it might be worth trying different combinations of layer sizes and types. Increasing the number of epochs and hidden layers would also likely further increase the model's accuracy, however it would also increase the amount of time and memory required to run the model.

Even though the temporal data (date and month) was removed from both datasets, the remaining data does likely reflect strong temporal patterns (seasonality, as well as changing climate conditions over the years). It might therefore be worth using a LSTM (long short-term memory) RNN model with this data in addition to the CNN model to compare predictions made.

LSTM (long short-term memory) model

Initial Parameters

epochs = 30

batch_size = 9

n_hidden = 4

timesteps = len(X_train[0])

input_dim = len(X_train[0][0])

n_classes = len(y_train[0])


```

model = Sequential()
model.add(LSTM(n_hidden, input_shape=(timesteps, input_dim)))
model.add(Dropout(0.5))
model.add(Dense(n_classes, activation='sigmoid'))

```

accuracy: 0.5629 - loss: 14.8939

Confusion Matrix

180/180 ————— 1s 2ms/step

Pred \ True	BASEL	MADRID
BASEL	3672	10
BELGRADE	1079	13
BUDAPEST	213	1
DEBILT	82	0
DUSSELDORF	29	0
HEATHROW	81	1
KASSEL	11	0
LJUBLJANA	59	2
MAASTRICHT	9	0
MADRID	451	7
MUNCHENB	8	0
OSLO	5	0
STOCKHOLM	4	0
VALENTIA	1	0

This model only recognized 2 stations.

Adjusting parameters

Changing the number of hidden layers from 4 to 16.

accuracy: 0.6467 - loss: 20.0309

The confusion matrix only showed 2 stations recognized again.

Keeping n_hidden at 16, changing the final activation function to tanh.

accuracy: 0.0063 - loss: 23.7586

The accuracy appears to decrease progressively with each epoch.

The confusion matrix showed that the model recognized only 3 stations, with most predictions being either München or Oslo.

Since the softmax activation function produced the best results for the CNN model, we can try applying it here, keeping the other parameters the same as before.

accuracy: 0.0576 - loss: 19.8599

The confusion matrix shows this model only recognized 2 stations – Madrid and Oslo.

Adding additional layers

Keeping all parameters the same as the previous round, adding the following Dense layer to the model structure, taken from the CNN model:

```
model.add(Dense(32, activation='relu'))
```

accuracy: 0.0965 - loss: 12993.6016

The confusion matrix shows this model recognized 3 stations – Belgrade, Budapest and Ljubljana.

Keeping the above structure, increasing batch_size to 18 and n_hidden to 64

accuracy: 0.0919 - loss: 9071.5908

The confusion matrix shows this model recognized 2 stations – Belgrade and Budapest.

Reducing the batch_size back to 9

accuracy: 0.0955 - loss: 38829.1602

Confusion matrix shows only 2 stations predicted – Basel and Belgrade.

Changed the size of the additional dense layer to 16 and the activation function to sigmoid.

accuracy: 0.0494 - loss: 38.0752

Confusion matrix shows only Belgrade and Madrid stations recognized.

Changing the optimizer function from rmsprop to adam

Also changed the activation function of the dense layer back to relu.

accuracy: 0.0908 - loss: 12005.5332

Confusion matrix still only shows the same 2 stations as above recognized.

Reset the optimizer back to rmsprop

Also changed the activation function of the dense layer to tanh.

accuracy: 0.0521 - loss: 19.7574

Confusion matrix shows only 2 stations (Madrid and Oslo) recognized.

Changing the dense layer activation function back to ReLU, adjusting n_hidden and the size of the dense layer to 64.

accuracy: 0.0968 - loss: 39928.0391

Once again, the confusion matrix shows the model only recognized 2 stations – Budapest and Ljubljana.

Changed the final activation layer function back to sigmoid, as it showed the highest level of accuracy.

accuracy: 0.6469 - loss: 62530.7656

The accuracy stabilized after 3 epochs while the loss continued to increase.

The confusion matrix shows that this model only predicted one location, Basel.

Observations

- All LSTM models took a long time to run, even with a relatively low number of epochs.
- The LSTM with the final activation layer set to sigmoid showed the highest level of accuracy.
- Adding additional dense layers, changing the final activation function or changing the optimizer function did not appear to affect the model's performance significantly.
- Overall, the LSTM did not perform as well with the weather and pleasant weather datasets as the CNN model did.
- One possible reason for this might be the fact that the weather data is unscaled. The larger variation in value weights might affect the results of the LSTM model more strongly than the CNN model.